

ich sehe leider immernoch formate wie

€89.92

es sollte aber 89,92€ zeigen

falls du mehr kontext hierfür benötigst sag bescheid

Das ist ein klassisches React-Phänomen! Wenn ein Regex im trockenen Test funktioniert, aber auf der Live-Seite nicht greift, liegt das bei React zu 99 % an einem von zwei Dingen:

1. **Das "Split-Node"-Problem:** Im HTML-Quellcode sieht es zwar wie `<span class="font-medium">€89.92</span>` aus, aber React rendert das intern oft als zwei getrennte Text-Nodes: eine Node für das €-Zeichen und eine Node für die Zahl 89.92. Da unser Skript jeden Text-Node einzeln prüft, findet es in der einen Node nur "€" und in der anderen nur "89.92". Der Regex sucht aber nach beidem zusammen und schlägt deshalb fehl.
2. **React's Re-Render-Zyklus:** React aktualisiert die Komponente, unser Skript ändert den Text, aber React bemerkt die DOM-Änderung, die es nicht selbst gemacht hat, und überschreibt sie im nächsten Bruchteil einer Sekunde wieder mit dem Original.

Wir gehen jetzt auf Nummer sicher und greifen tiefer in die Trickkiste. Wir passen die Währungs-Erkennung so an, dass sie auch greift, wenn das Euro-Zeichen in einer separaten Node steht oder ganz fehlt (da wir wissen, dass es um Preise geht).

Tausche die Währungs-Erkennung in der Funktion `processTextNode` gegen diesen robusteren Code-Block aus:

JavaScript

```
// --- B. WÄHRUNGSFORMAT ANPASSEN (BOMBENSICHER) ---
// Fall 1: Das €-Zeichen und die Zahl sind im selben Text-Node (wie bisher)
// Wir erlauben auch Tausendertrennzeichen (z.B. €1,089.92 -> 1.089,92€)
const combinedCurrencyRegex = /€\s*([\d,]+\.\d{2})/g;
if (combinedCurrencyRegex.test(text)) {
  text = text.replace(combinedCurrencyRegex, (match, ganzzahl, dezimal) => {
    // Kommas in Tausendern zu Punkten machen
    const saubereGanzzahl = ganzzahl.replace(/,/g, '.');
    return `${saubereGanzzahl}.${dezimal}€`; // Komma und € am Ende
  });
}
```

```

// Fall 2: React hat die Node gesplittet und wir sehen NUR die Zahl (z.B. "89.92")
// Wir prüfen, ob die Node exakt eine Preis-Zahl ist (zwei Nachkommastellen)
else {
  const standaloneNumberRegex = /^(d+)\.(\d{2})$/;
  if (standaloneNumberRegex.test(text.trim())) {
    // Wir prüfen, ob das übergeordnete Element einen Hinweis auf Währung gibt
    // oder ob das vorherige Element ein € war.
    const parentText = node.parentNode ? node.parentNode.textContent : "";
    if (parentText.includes('€')) {
      text = text.replace(standaloneNumberRegex, '$1,$2€');

      // Optional: Das alte, alleinstehende €-Zeichen im Parent verstecken,
      // da wir es jetzt hinter die Zahl gepackt haben.
      if (node.previousSibling && node.previousSibling.nodeValue &&
node.previousSibling.nodeValue.includes('€')) {
        node.previousSibling.nodeValue = node.previousSibling.nodeValue.replace('€', "");
      }
    }
  }
}
}
}

```

Wenn du diesen Codeblock einfügst, fangen wir beide React-Szenarien ab.

Lass uns herausfinden, was React hier im Hintergrund genau treibt: Kannst du einmal mit der rechten Maustaste auf die €89.92 auf deiner Testseite klicken, "Untersuchen" (Inspect) wählen und mir sagen, ob das Euro-Zeichen und die Zahl wirklich im exakt selben HTML-Tag stehen, oder ob dort vielleicht noch versteckte -Tags dazwischen sind?